```
const BlindAuction = artifacts.require('../contracts/BlindAuction.sol')
const truffleAssert = require('truffle-assertions');
```

The next step is initializing the numerous variables of the blind auction. You need to set up a minimum number of bidders and the bid amounts:

```
  const success = '0x01'
  let blindAuction
  const onlyBeneficiaryError = 'Only beneficiary can perform this action'
  const validPhaseError = 'Invalid phase'
  const badRevealError = 'Not matching bid'

  // Bidding amount placeholders in ether for fast modification.
  let BID1 = 1
  let BID2 = 2
  let BID3 = 3

  // Account placeholders for user accounts for testing
  let BEN = accounts[0]
  let ACC1 = accounts[3]
  let ACC2 = accounts[4]
  let ACC3 = accounts[5]
```

After these data definitions, you’re ready to code the test functions (listing 10.4). Be warned, these functions are much more complex than those for the counter and ballot smart contracts; blind auction represents a practical application, including encryption and hashing.

Listing 10.4 blindAuctionTest.js

```
  beforeEach('Setup contract for each test', async function () {

  describe('Initialization and Phase Change.', async ()=>{
    it('Success on initialization to bidding phase.',async function() {...
    it('Success on phase change by beneficiary.', async function() {...
       it('Success on change from DONE phase to INIT phase.',
                                                 async function()
       ...}

  describe('Bidding Phase.', async ()=>{
it('Success on single bid.', async function () { ...
```